

JavaScript: The New Parts

JavaScript has come a long way in its 20 odd years of existence. It continues to grow from strength to strength because of its versatility. ES6 adds a new dimension to JavaScript, taking it to the next level. This article is the sixth in this series.

In this part we cover destructuring. This feature reduces many lines of code and also improves readability.

Destructuring can also be used during declaration, initialisation, iteration and in function parameters.

Destructuring is about accessing one or more elements from a collection in an easier manner. It can be used with arrays and objects, which are collections to store sets of values with a variable name. Accessing values in an array is done by using an index. To access values from an object, we use the property name.

Destructuring objects

Given below is an object to store details of one Linux distribution - Linux Mint. Instead of `var`, we defined the structure as a `const`, which is a new keyword in ES6 (covered in an earlier part of this series).

```
1. const distro = {
2.   name: 'Linux Mint',
3.   os_type: 'linux',
4.   based_on: ['debian', 'ubuntu'],
5.   origin: 'Ireland',
6.   arch: ['i386', 'x86_64'],
7.   popularity: 1
8. };
9. let { name, origin } = distro;
```

Let us say that, in our program, we only need the name of the Linux distribution and country of origin. I can use the destructuring feature to extract only these two properties from the whole big object.

In Line 9, the object property name and local variable name are chosen as the same. Using another feature of ES6, we can further reduce Line 9 to the one below.

```
9. let { name, origin } = distro;
```

Destructuring can be applied to even array elements and nested object properties. To extract the first element of an architecture type, we can use the following code:

```
let { arch: [first]} = distro;
console.log('Arch: ${first}'); //Arch: i386
```

Destructuring arrays

Array elements are accessed based on their index position. Using destructuring, we can select subsets of an array and assign them to new variables. In the example below, we

split the UTC date string into an array of strings. We extract elements of interest, which are the date, month and year.

```
1. let today = new Date();
2. let UTCstring = today.toUTCString();
3. let [, date, month, year] =
4.     today.toUTCString().split(" ");
5.
6. console.log("date: " + date); // date: 16
7. console.log("month: " + month); // month: Jan
8. console.log("year: " + year); // year: 2016
```

The UTC string contains 'Mon, 18 Jan 2016 14:24:09 GMT'. After the split, it is an array of strings [Mon, 18, Jan, 2016, 14:25:06, GMT]. The assignment operation in Line 3 extracts only the date, month and year. Note that the first element (index 0) is the week day, which is skipped with a comma in the beginning.

A two-dimensional matrix

Picking elements selectively from a two-dimensional matrix can be done in a single assignment operation.

```
let matrix = [[1, 2, 3],[4, 5, 6],[7, 8, 9]];
let [[a11],[a22],[a33]] = matrix;
console.log("Diagonal: " + a11, a22, a33);
```

```
Output:
Diagonal: 1 5 9
```

Swapping elements

Another interesting variation of an assignment is the swapping of elements without the use of a temporary variable:

```
let [a, b] = [1, 2];
[a, b] = [b, a]; // elements swapped
```

A more interesting variant is to do a transpose of a two-dimensional matrix:

```
let matrix = [[1, 2],[3, 4]];
console.log(matrix); // 1, 2, 3, 4

let [[a11,a12],[a21,a22]] = matrix;
[[a11,a12],[a21,a22]] = [[a11, a21],[a12, a22]];
console.log(a11, a12, a21, a22); // 1, 3, 2, 4
```